

This is the command center for **Escape from Mars**. By separating your logic, you ensure that the biological and chemical systems are automated and ticking every frame, while Evan and Roberto can plug their specific modules into your shared state.

Here is your **main.py** starter, followed by the final automated roadmap to get you to 9:00 PM.

The Main Loop: **main.py**

This script initializes the **Project Return** parameters and runs the core simulation.

```
Python
import pygame
import sys
from systems import EscapeFromMarsSystem
# import evan_physics # Mechanics & Physics Lead [cite: 8]
# import roberto_view # Environment & Aesthetics Lead [cite: 12]

def main():
    pygame.init()

    # 1. Initialize the shared state and systems
    # Michael: HUD, Sabatier, and Bio-Decay [cite: 16, 17, 19]
    mars_game = EscapeFromMarsSystem()

    # Screen Setup (Placeholder size)
    screen = pygame.display.set_mode((1280, 720))
    pygame.display.set_caption("Escape from Mars: Sol 559")
    clock = pygame.time.Clock()

    # 2. Global Countdown Timer [cite: 11]
    # Set for the Trans-Earth Injection window
    launch_window_timer = 5400 # 90 minutes in seconds

    while mars_game.is_alive:
        # Calculate Delta Time (seconds passed since last frame)
        dt = clock.tick(60) / 1000.0
        launch_window_timer -= dt

        # Event Handling
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
```

```

# Michael: Trigger Sabatier via UI click [cite: 17, 21]
if event.type == pygame.MOUSEBUTTONDOWN:
    # Placeholder for UI button check
    mars_game.run_sabatier_process()

# 3. Michael's Automated Logic [cite: 16]
# Ticks down bone density and radiation constantly [cite: 21]
mars_game.update_bio_decay(dt)

# 4. Evan's Automated Logic [cite: 8]
# Implement 38% gravity (3.721 m/s²) [cite: 9, 21]
# evan_physics.apply_gravity(dt, mars_game.bone_density)

# 5. Roberto's Automated Logic [cite: 12]
# Render the Jezero Crater and Starship [cite: 13, 15]
# If timer is low, trigger Blue Sunset Rayleigh scattering
# roberto_view.render(screen, mars_game, launch_window_timer)

pygame.display.flip()

print("Mission Failure: Biological limits exceeded.")

if __name__ == "__main__":
    main()

```

Automated 60-Minute Roadmap (Final Sprint)

To hit the **9:00 PM** deadline, follow this parallel-track roadmap.

Time	Michael (Systems/UX)	Evan (Mechanics)	Roberto (Aesthetics)
8:00 - 8:20	Mass-Limit Lock: Finalize the inventory code so scientific samples increase fuel needs.	Gravity Lock: Ensure jumping is 2.6x higher/slower than Earth.	Environment Lock: Finish the Jezero Crater landscape and "Leopard Spot" rocks.

8:20 - 8:40	HUD Overlay: Create the wrist-display for Fuel, Oxygen, and Battery.	Ignition Sequence: Code the 3-stage launch logic (Power $\rightarrow$ Prime $\rightarrow$ Launch).	Lighting Automation: Link the "Blue Sunset" visual to the 5-minute timer mark.
8:40 - 8:55	Integration: Verify that <code>bone_density</code> affects Evan's speed and Roberto's HUD.	Win/Loss Logic: Ensure the ship only launches if <code>Methane >= 100</code> .	Ship Texturing: Ensure the Stainless Steel Starship looks "ready" when fueled.
8:55 - 9:00	DEPLOY: Final bug pass.	DEPLOY: Check build.	DEPLOY: Audio muffling check.

Scientific Integration Notes

- **Fuel Chemistry:** Ensure the `run_sabatier_process` logic in `systems.py` correctly represents $\text{CO}_2 + 4\text{H}_2 \rightarrow \text{CH}_4 + 2\text{H}_2\text{O}$ to provide the "valuable learning" edge.
- **Low-G Movement:** Evan must use $3.721 \sim \text{m/s}^2$ as the core gravity constant to maintain scientific accuracy.
- **Visual Atmosphere:** The shift to a **Blue Sky** sunset must be tied to the Rayleigh scattering effect in thin Martian dust.